



Transcript Of Individual Presentation: Neural Network Models for Object Recognition (Track 2)

Cover Slide

Hi, Good Evening everyone “**Neural Network Models in Object Recognition.**”

In this presentation, I’ll walk you through the process of designing, training, and evaluating deep learning models—specifically a **Custom CNN** and a **Transfer Learning model (MobileNetV2)**—for object recognition using the **CIFAR-10 dataset**.

Slide 1: Introduction

This project explores two major approaches to deep learning for image classification:

1. Building a **custom CNN model** from scratch.
2. Applying **transfer learning** using a pretrained **MobileNetV2** model on ImageNet.

Both models were evaluated based on their accuracy in classifying images into ten object categories.

The real-world relevance of such systems is vast—ranging from **facial recognition** to **autonomous driving** and **smart surveillance**.

The **CIFAR-10 dataset** was selected due to its moderate size, diversity, and common use in academic research (Krizhevsky, 2009).

The implementation was carried out using **TensorFlow**, **Keras**, and various visualization libraries for analysis and evaluation.

Slide 2: Exploratory Data Analysis (EDA)

Before model building, an **Exploratory Data Analysis (EDA)** was conducted to verify that the dataset was balanced and suitable for training.

CIFAR-10 contains **60,000 color images** (32×32 pixels, RGB), equally divided into **10 classes**: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck (Krizhevsky, 2009).

Data visualization confirmed balanced class distribution—essential for fair model evaluation and unbiased training.

Random samples from each class showed **intra-class variability** (e.g., different car angles and lighting), which adds complexity but also improves model generalization.

Slide 3: Preprocessing

Preprocessing is a vital step in computer vision workflows. The key preprocessing steps included:

1. **Normalization:** Pixel values scaled between 0–1 to stabilize and accelerate training (Brownlee, 2022).
 2. **Label Encoding:** Used sparse categorical encoding for efficient multi-class handling with TensorFlow.
 3. **Image Resizing:** CIFAR-10 images were resized to **96×96** pixels to match MobileNetV2 input requirements (Howard et al., 2017).
 4. **Shuffling and Augmentation:** Horizontal flips and random crops were applied to increase data variability and minimize overfitting.
-

Slide 4: Data Partitioning

The dataset was split into:

- **40,000 images** for training
- **10,000 images** for validation
- **10,000 images** for testing

This **three-way split** ensures reliable performance monitoring and early detection of overfitting or underfitting.

The **validation set** guides hyperparameter tuning, while the **test set**—kept completely unseen—serves as the final benchmark for real-world model performance (TensorFlow Docs, 2023).

Slide 5: Custom CNN Architecture

The first model was a **Custom Convolutional Neural Network**, built using the **Keras Sequential API**.

Architecture overview:

- Two **Conv2D layers** with ReLU activation
 - **MaxPooling** layers for spatial reduction
 - **Flatten layer** to convert feature maps into vectors
 - **Dense layer (128 neurons)** followed by **Dropout (0.5)**
 - Final **Dense(10)** layer with **Softmax** activation
-

This design captures low- to mid-level features (edges, corners, textures) while Dropout prevents overfitting. Although simple, it provides a solid **baseline** for comparison with more advanced models.

Slide 6: Transfer Learning with MobileNetV2

The second approach used **Transfer Learning** with **MobileNetV2**, pretrained on **ImageNet** (Howard et al., 2017; Deng et al., 2009).

Steps included:

1. Loading MobileNetV2 without its top classification layers.
2. Freezing base layers during initial training.
3. Adding **Global Average Pooling**, **Dense(128)**, **Dropout**, and **Dense(10)** with **Softmax**.
4. Unfreezing the last 50 layers for fine-tuning to CIFAR-10.

This method drastically reduces training time and computational cost while maintaining high accuracy—ideal for **mobile or edge AI applications**.

Slide 7: Training Process

Training was optimized for efficiency and generalization:

- **Optimizer:** Adam, for adaptive learning and faster convergence.
- **Loss Function:** Sparse Categorical Crossentropy for multi-class classification.
- **Batch Size:** 64, balancing performance and memory use.
- **Epochs:**
 - CNN: 20 epochs.
 - MobileNetV2: Two phases—10 frozen + 10 fine-tuned epochs.

Early stopping was implemented to halt training when validation performance stopped improving, preventing overfitting and saving resources.

Consistent logging of accuracy, loss, and training time ensured reproducibility and transparent performance tracking.

Slide 8: Model Evaluation – Accuracy and Loss

After training, both models were evaluated using accuracy and loss metrics.

- **Custom CNN:** Achieved around **72% accuracy**. Training showed signs of mild overfitting after the 6th epoch.
- **MobileNetV2:** Achieved around **85% accuracy** with faster convergence and better generalization.

The **learning curves** indicated smoother and more stable performance in MobileNetV2 due to pretrained ImageNet weights (Deng et al., 2009; Chollet, 2017). This confirms the strong advantage of **transfer learning** for small datasets.

Slide 9: Confusion Matrix Analysis

The **confusion matrix** provided detailed insight into model performance.

MobileNetV2 showed strong diagonals, indicating high accuracy for classes like **airplane, ship, and automobile**.

However, misclassifications occurred between visually similar categories such as:

- Cat ↔ Dog
- Deer ↔ Horse
- Bird ↔ Frog

These errors often stem from low-resolution images or overlapping visual features (Krizhevsky, 2009).

The confusion matrix also highlighted trade-offs between precision and recall across classes—valuable for refining models for domain-specific applications such as **security** or **healthcare**.

Slide 10: Single Image Test

To simulate a real-world use case, MobileNetV2 was tested on single, unseen images from the test set.

The model correctly predicted most images with **80–90% confidence**, demonstrating strong real-time inference capability.

Even incorrect predictions had the correct class as the **second-highest** probability—supporting future exploration of **top-k accuracy** and **confidence-based decisions**.

This experiment bridged the gap between **experimental training** and **practical deployment**, showing MobileNetV2's potential for **edge AI and mobile applications** (Howard et al., 2017).

Slide 11: Strengths and Weaknesses

Strengths:

- Transfer learning greatly improves accuracy and reduces training time.
- MobileNetV2 is lightweight and optimized for mobile devices.
- Both models generalize well even on limited data.

Weaknesses:

- Limited flexibility in modifying MobileNetV2 architecture.
 - Resizing CIFAR-10 images may cause minor interpolation artifacts.
 - Custom CNN can overfit without strong regularization methods.
-

Slide 12: Code Implementation

All implementation was done in **Python**, using **TensorFlow** and **Keras**.

Main components:

1. Dataset loading and preprocessing
2. Model setup for CNN and MobileNetV2
3. Training callbacks (e.g., early stopping)
4. Evaluation scripts for accuracy/loss plots, confusion matrices, and single-image testing

The codebase is modular, clean, and reusable—making it suitable for future real-world deployment or further research.

Slide 13: Conclusion

In summary, this project demonstrates that **deep learning**, especially when combined with **transfer learning**, can deliver powerful results in **object recognition** tasks.

MobileNetV2 outperformed the **custom CNN**, showing higher accuracy and faster convergence.

Regularization techniques like **dropout**, **validation monitoring**, and **early stopping** were crucial for building reliable, generalizable models.

Overall, the findings confirm that **transfer learning is a highly effective approach** for small datasets, and **CIFAR-10** remains an excellent benchmark for evaluating image classification models.

References

Brownlee, J. (2022). How to Develop a CNN From Scratch for CIFAR-10 Photo Classification.

Machine Learning Mastery.

<https://machinelearningmastery.com/how-to-develop-a-cnn-from-scratch-for-cifar-10-photo-classification/> (Accessed: 11 July 2025)

Chollet, F. (2017). Deep Learning with Python. Manning Publications. Available at:

<https://sourestdeeds.github.io/pdf/Deep%20Learning%20with%20Python.pdf> (Accessed: 11 July 2025)

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A Large-Scale Hierarchical Image Database. IEEE Conference on Computer Vision and Pattern Recognition

(CVPR). Available at:

<https://ieeexplore.ieee.org/document/5206848> (Accessed: 12 July 2025)

Howard, A. G., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861. <https://arxiv.org/abs/1704.04861> (Accessed: 12 July 2025)

Keras CIFAR-10 example. Available at: <https://keras.io/api/datasets/cifar10/> (Accessed: 13 July 2025)

Krizhevsky, A. (2009). Learning Multiple Layers of Features from Tiny Images. University of Toronto. Available at: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf> (Accessed: 13 July 2025)

TensorFlow/Keras Documentation. Transfer learning and fine-tuning guide. https://www.tensorflow.org/tutorials/images/transfer_learning (Accessed: 13 July 2025)